

Deep Learning & Generative AI

Complete Exam Reference Guide

Formulas | Worked Examples | Sample Exam Paper | Cheat Sheet

IIT Bombay | ePGD AI & Data Science

End Semester | May 2026 | Open Book | 50 Marks | 3 Hours

Source Materials

01_Detailed_Concepts_and_Math.pdf | 03_Mock_Paper_with_Solutions.pdf
DL_FormulaGuide | DL_Part3_SampleExams | DL_Part4_MasterCheatSheet | ePGD_End_term_Model_Paper

Exam Overview & Weightage	4
Part 1: Formula Index	5
A. N-gram Language Models & Smoothing	5
A1 — N-gram Probability (Chain Rule + Markov Assumption)	5
A2 — Laplace (Add-k) Smoothing.....	5
A3 — Perplexity.....	5
B. Word Embeddings (word2vec & GloVe).....	5
B1 — Skip-gram Negative Sampling (word2vec SGNS)	6
B2 — GloVe Objective.....	6
C. RNN / LSTM / GRU	6
C1 — Vanilla RNN Hidden State.....	6
C2 — LSTM (Full Forward Pass — 4 Gates)	6
C3 — GRU (Simpler Alternative: 2 gates, ~25% fewer params)	7
C4 — RNN vs LSTM vs GRU Comparison	7
D. Seq2Seq + Bahdanau Attention.....	7
D1 — Bahdanau Attention (Additive)	7
D2 — Luong Attention Scoring Functions	7
E. The Transformer	8
E1 — Scaled Dot-Product Self-Attention.....	8
E2 — Multi-Head Attention (MHA)	8
E3 — Positional Encoding (Sinusoidal — original Transformer).....	8
E4 — Causal Masking (Decoder).....	9
E5 — Feed-Forward Network (FFN) — Per Position.....	9
E6 — Transformer Parameter Count	9
F. Pre-training Objectives & Fine-tuning	9
F1 — LoRA (Low-Rank Adaptation) — PEFT	9
G. RLHF & Alignment.....	10
G1 — Reward Model Training (Bradley-Terry).....	10
G2 — RLHF Regularized Objective (KEY FORMULA).....	10
G3 — PPO-CLIP Surrogate Objective	10
G4 — RL Framework for LLMs	10
H. CNN: Dropout, Batch Normalization & Attention	11
H1 — Batch Normalization.....	11
H2 — SE Block (Squeeze-and-Excitation / Channel Attention)	11
H3 — Dropout Variants for CNNs	11
I. Image Segmentation	11
J. Agentic Frameworks & RAG	12
J1 — RAG Variants	12
Part 2: Worked Examples	13
Simple Level Examples	13
S1 — Bigram Probability Calculation	13

S2 — Perplexity Calculation	13
S3 — LSTM Gate Numerical Calculation	13
S4 — Bradley-Terry Reward Probability	14
Medium Level Examples	14
M1 — Scaled Self-Attention Numerical (Full Walkthrough)	14
M2 — Batch Normalization Forward Pass.....	14
M3 — Bahdanau Attention Worked Example	15
Complex Level Examples	15
C1 — BERT-base Parameter Count (Full Derivation)	15
C2 — RLHF Optimal Policy Derivation.....	15
C3 — PPO-CLIP: Gradient Behavior Analysis	16
Section 1 — Short Answers / MCQ / True-False [20 × 1 = 20 Marks]	17
Section 2 — CNN Regularisation [5 Marks]	19
Section 3 — Attention [5 Marks]	20
Section 4 — Transformer Details [6 Marks]	21
Section 5 — Pre-training & Fine-tuning [4 Marks]	21
Section 6 — Prompting & In-Context Learning [2 Marks]	22
Section 7 — Instruction Tuning & Alignment [4 Marks]	22
Section 8 — RLHF Mathematics [5 Marks]	22
Section 9 — Algorithms (PPO) [4 Marks]	23
Section 10 — Retrieval & Agents [3 Marks]	24
Section 11 — Design Problem [6 Marks]	24
Part 4: Last-Hour Cheat Sheet.....	26
Formula Quick Reference.....	26
Most Common Exam Traps.....	26
Architecture Quick Reference.....	27
Key Numerical Benchmarks	27
Answer Templates for Exam Questions.....	27
True/False Template.....	27
MCQ Template	27
Derivation/Explanation Template	27
Design Question Template	27

Exam Overview & Weightage

WEIGHTAGE: LLMs / Transformers / RLHF / Prompting / NLP → 70% | CNN / Segmentation / DL Basics → 30%

FORMAT: Open Book | 50 Marks | 3 Hours | 11 Sections | Section 1: 20 MCQ/T-F (1 mark each), Sections 2-11: Short answer & calculations

HOW TO USE THIS GUIDE: Formula Index → understand every formula with plain English. Worked Examples → step-by-step solutions at 3 levels. Sample Exam → practice with full solutions. Cheat Sheet → last-hour review.

Part 1: Formula Index

Every important formula — explained for beginners. Each entry: Formula → Symbols → Plain English → Intuition.

A. N-gram Language Models & Smoothing

A1 — N-gram Probability (Chain Rule + Markov Assumption)

General N-gram Probability

$$P(w_1, w_2, \dots, w_N) = \prod P(w_i \mid w_{\{i-n+1\}}, \dots, w_{\{i-1\}})$$

Bigram: $P(w_i \mid w_{i-1}) = \text{Count}(w_{i-1}, w_i) / \text{Count}(w_{i-1})$

Symbol	Meaning	Plain English
$P(w_i \mid \text{context})$	Conditional probability of word w_i	What is the chance of seeing this word after the previous words?
$\text{Count}(w_1, w_2)$	Frequency of bigram in training corpus	How many times did these two words appear together?
n	Order of n-gram (bigram=2, trigram=3)	How many words of history are used

Plain English: To predict the next word, count how often word-pairs appeared in training. If 'the cat' appeared 20 times and 'the' appeared 100 times, $P(\text{cat} \mid \text{the}) = 0.20$.

A2 — Laplace (Add-k) Smoothing

Prevents zero probabilities for unseen n-grams

$$P_{\text{smooth}}(w_i \mid w_{i-1}) = [\text{Count}(w_{i-1}, w_i) + k] / [\text{Count}(w_{i-1}) + k \cdot |V|]$$

$k=1 \rightarrow$ Laplace smoothing; $k < 1 \rightarrow$ Add-k smoothing

Symbol	Meaning	Plain English
k	Smoothing constant (usually 1)	Pretend every n-gram appeared k extra times
$ V $	Vocabulary size (total unique words)	How many different words exist in our dictionary

Why needed: Without smoothing, one unseen word pair gives probability=0, making the entire sentence probability=0. Adding k to every count ensures nothing is impossible.

A3 — Perplexity

How 'surprised' is the model? Lower = Better.

$$PP(W) = \exp \left(- (1/N) * \sum \log P(w_i \mid w_{\{<i\}}) \right)$$

Symbol	Meaning	Plain English
$PP(W)$	Perplexity score (lower is better)	How many equally likely words the model considers at each step
N	Number of tokens in test corpus	Length of the test text
$P(w_i \mid \dots)$	Model probability for each token	How confident the model is about each word

Interpretation: $PP=10$ means model acts as if choosing from 10 options at each step. $PP=1$ is perfect. Random model on 50,000-word vocab has $PP=50,000$. Lower is always better.

B. Word Embeddings (word2vec & GloVe)

B1 — Skip-gram Negative Sampling (word2vec SGNS)

Train by distinguishing true context words from noise

```

$$L(c,o) = -\log \sigma(uo.vc) - \sum_{n=1}^K \log \sigma(-un.vc)$$


$$\sigma(x) = 1 / (1 + \exp(-x))$$
 [sigmoid function]
```

Symbol	Meaning	Plain English
vc	Focus (center) word vector	Vector representation of the word we are training on
uo	True context word vector	Vector of a word that actually appears near the focus word
un	Noise (negative sample) vector	Vector of a random word unlikely to be in context
K	Number of negative samples	How many random 'wrong' words to contrast against (typically 5-20)
sigma	Sigmoid: squishes to (0,1)	Turns any number into a probability between 0 and 1

Plain English: Train a true/false classifier: 'Is this word a real neighbor of the focus word?'
Push focus and context words together; push focus and noise words apart. This builds meaning into vectors.

B2 — GloVe Objective

Factorize the global co-occurrence matrix — NO negative sampling

```

$$J = \sum_{i,j} f(X_{ij}) * (w_i.w_j\_tilde + b_i + b_j\_tilde - \log X_{ij})^2$$


$$f(x) = \min(1, (x/x\_max)^\alpha)$$
 weighting; x_max=100, alpha=0.75
```

KEY EXAM FACT: GloVe uses NO negative sampling. It directly minimises weighted squared error on log co-occurrence counts. This is the most common exam trap — do not confuse with word2vec SGNS.

C. RNN / LSTM / GRU

C1 — Vanilla RNN Hidden State

Memory update at each timestep

```

$$h_t = \tanh(W_{xh} * x_t + W_{hh} * h_{t-1} + b)$$

```

Symbol	Meaning	Plain English
h_t	Hidden state at time t	The model's 'memory' after processing word t
h_{t-1}	Previous hidden state	Everything remembered up to word t-1
x_t	Input at timestep t (word embedding)	The current word, represented as numbers
W_{xh}, W_{hh}	Learnable weight matrices	The model's parameters — same weights used at every step
$\tanh$	Activation function, outputs in (-1,1)	Squishes the result to a bounded range for stability

Vanishing Gradient: The gradient product shrinks by $\tanh'(\cdot) \leq 1$ at every timestep. Over 50+ steps the gradient effectively becomes zero — RNN forgets long-range dependencies.

C2 — LSTM (Full Forward Pass — 4 Gates)

4 gates give LSTM long-term memory (constant-error carousel)

```
ft = sigma(Wf.[h_{t-1}; xt] + bf)    <- Forget Gate: what % of old memory to keep
it = sigma(Wi.[h_{t-1}; xt] + bi)    <- Input Gate: how much new info to write
ct_hat = tanh(Wc.[h_{t-1}; xt] + bc) <- Candidate: proposed new memory content
ot = sigma(Wo.[h_{t-1}; xt] + bo)    <- Output Gate: what to expose from memory
Ct = ft o C_{t-1} + it o ct_hat        <- Cell State (long-term memory update)
ht = ot o tanh(Ct)                    <- Hidden State (output this step)
```

Symbol	Meaning	Plain English
ft in [0,1]	Forget gate	0=erase memory, 1=keep everything from previous cell state
it in [0,1]	Input gate	How much of the new candidate info to actually write into memory
Ct	Cell state (long-term memory)	The LSTM 'RAM' — can store info for 1000+ steps
ht	Hidden state (short-term output)	What the LSTM 'says' at this step; passed to next step & layer
o	Element-wise (Hadamard) multiplication	Multiply corresponding elements of two vectors

Why LSTM fixes vanishing gradient: $dC_t/dC_{t-1} = \text{diag}(f_t)$. When $f_t \sim 1$, this is the identity matrix — gradient flows back undamped. Compare to RNN where every step multiplies by $\tanh'(\cdot) \leq 1$.

C3 — GRU (Simpler Alternative: 2 gates, ~25% fewer params)

2 gates instead of 3 — comparable performance to LSTM

```
rt = sigma(Wr.[h_{t-1}; xt])      <- Reset Gate
zt = sigma(Wz.[h_{t-1}; xt])      <- Update Gate
ht_hat = tanh(W.[rt*ht_{t-1}; xt]) <- Candidate hidden state
ht = (1-zt)*ht_{t-1} + zt*ht_hat   <- Final hidden state (blended)
```

C4 — RNN vs LSTM vs GRU Comparison

Property	Vanilla RNN	LSTM	GRU
Gates	0	3 (f, i, o)	2 (r, z)
Cell State	None	Yes (Ct)	None
Long-Range Deps	Poor (vanishing)	Strong	Strong
Params/layer	$n(m+n+1)$	$4n(m+n+1)$	$3n(m+n+1)$
Use Today	Rarely	Streaming/time-series	Small data, fast training

D. Seq2Seq + Bahdanau Attention

D1 — Bahdanau Attention (Additive)

3-step process: score -> normalize -> blend

```
e_{ij} = va^T . tanh(Wa*s_{i-1} + Ua*hj)  <- alignment score (scalar)
alpha_{ij} = exp(e_{ij}) / sum_k exp(e_{ik}) <- softmax -> weights sum to 1
ci = sum_j alpha_{ij} * hj                  <- context vector (weighted sum)
```

Symbol	Meaning	Plain English
s_{i-1}	Previous decoder hidden state (query)	What the decoder currently 'wants' / is looking for
h_j	Encoder hidden state at position j (key)	What source word j 'means' in full context
e_{ij}	Raw alignment score (scalar)	How relevant is source word j when generating output word i?
α_{ij}	Attention weight	What fraction of attention goes to source word j (sums to 1)
c_i	Context vector	Weighted blend of all encoder states — fresh info for each decoder step

Key Insight: Attention is MANY-TO-MANY (soft alignment). Multiple decoder steps can attend to the same encoder position; encoder positions can receive zero attention. There is NO one-to-one constraint.

D2 — Luong Attention Scoring Functions

Three variants — dot product is most common in practice

```

score(st, hj) = st^T . hj                                <- dot (simplest)                = st^T
. W . hj        <- general (learnable W)              = v^T . tanh(W[st;hj])
<- concat (similar to Bahdanau)

```

	Bahdanau	Luong
Decoder State Used	$s_{\{t-1\}}$ (PREVIOUS)	st (CURRENT)
Score Type	Additive (tanh)	Dot / General / Concat
Context Used	fed into decoder RNN	mixed after RNN: $\tilde{s} = \tanh(Wc[ct;st])$

E. The Transformer

E1 — Scaled Dot-Product Self-Attention

The core operation of every Transformer layer

```

Attention(Q, K, V) = softmax( Q*K^T / sqrt(dk) ) * V
Per-position: al = sum_t alpha_{lt} * vt   where alpha_{lt} = softmax_t(ql.kt / sqrt(dk))

```

Symbol	Meaning	Plain English
Q (Query)	What each token is looking for (n x dk)	Each row = one token's 'search query'
K (Key)	What each token offers as identifier (n x dk)	Each row = one token's 'label' for matching
V (Value)	Actual content each token contributes (n x dv)	Each row = the information payload that gets passed if selected
$Q \cdot K^T$	Similarity matrix (n x n)	How much does each token's query match each other token's key?
$\sqrt{d_k}$	Scaling factor	Prevents giant dot products that push softmax to saturate and kill gradients
$\alpha_{\{l\}}$	Attention weight from position l to t	How much position l attends to position t
al	Output at position l	Weighted sum of value vectors — 'retrieved information' for token l

Why $\sqrt{d_k}$? Q.K entries have variance d_k (sum of d_k unit-variance products). Without scaling, large $d_k \rightarrow$ huge dot products $\rightarrow$ very peaked softmax $\rightarrow$ near-zero gradients for all non-max positions. Dividing by $\sqrt{d_k}$ keeps variance ~ 1 .

E2 — Multi-Head Attention (MHA)

Run h independent attention computations in parallel

```

head_i = Attention(Q*W_Qi, K*W_Ki, V*W_Vi)   for i=1...h  MHA(X) = Concat(head_1,
..., head_h) * W_O
W_Qi, W_Ki, W_Vi in R^(d x d/h)   (same total parameter budget as single-head)

```

Why multiple heads? Softmax forces a single head to be 'peaky' — it can only capture one dominant relationship. h heads in different subspaces simultaneously: Head 1 might learn subject-verb agreement, Head 2 coreference, Head 3 positional offsets.

E3 — Positional Encoding (Sinusoidal — original Transformer)

Gives the model information about token order

```

PE(pos, 2i)   = sin( pos / 10000^(2i/d) )  PE(pos, 2i+1) = cos( pos / 10000^(2i/d) )
)

```


PE Variant	Strength	Weakness
Sinusoidal	No params; extrapolates in principle	Fixed; only absolute, not inherently relative
Learned Absolute (BERT/GPT-2)	Flexible; optimised end-to-end	Cannot extrapolate beyond training length
RoPE (LLaMA)	Dot product depends only on i-j (true relative)	Slightly more compute; some saturation at very long offsets
ALiBi (BLOOM)	No params; excellent extrapolation	Different mechanism; no relative encoding in the vectors

E4 — Causal Masking (Decoder)

Prevent decoder from 'seeing the future' during training

Set $e_{\{l,t\}} = -\text{infinity}$ for all $t > 1$ (before softmax) Mask matrix (T=4):

```
[[
0,  -inf, -inf, -inf ], [ 0,    0,  -inf, -inf ], [ 0,    0,    0,  -inf ], [ 0,
0,    0,    0  ]]
```

Plain English: During training we feed the entire target sequence at once. But position 3 must NOT know position 4 (cheating). Setting those scores to -infinity makes softmax give them exactly 0 weight. This preserves the auto-regressive property.

E5 — Feed-Forward Network (FFN) — Per Position

Applied independently at each position after attention

$\text{FFN}(x) = \max(0, x \cdot W_1 + b_1) \cdot W_2 + b_2$ W_1 in $\mathbb{R}^{(d \times 4d)}$, W_2 in $\mathbb{R}^{(4d \times d)}$
[expansion ratio = 4]

Role: Attention MIXES information across positions. The FFN TRANSFORMS it at each position independently. Most factual 'world knowledge' in a Transformer is stored in FFN weights.

E6 — Transformer Parameter Count

For one encoder/decoder block (ignore biases & LayerNorm)

MHA params = $4 \cdot d^2$ (W_Q, W_K, W_V, W_O each $d \times d$) FFN params = $2 \cdot 4d \cdot d = 8d^2$ ($W_1: d \times 4d, W_2: 4d \times d$) Per-layer total $\sim 12d^2$ BERT-base: $d=768$, 12 layers $\rightarrow 12 \times 12 \times 768^2 \sim 85\text{M} + \text{embeddings} \sim 110\text{M}$ total

F. Pre-training Objectives & Fine-tuning

Model	Architecture	Pre-training Objective	Best For
BERT	Encoder-only	MLM (mask 15% tokens, predict) + NSP	Classification, NER, extractive QA
GPT	Decoder-only	Causal LM (predict next token)	Generation, completion, instruction following
T5	Encoder-Decoder	Span corruption with sentinel tokens (<extra_id_0>)	Seq2Seq: translation, summarisation
BART	Encoder-Decoder	Multiple noising (deletion, masking, sentence permutation)	Summarisation, generation, translation

F1 — LoRA (Low-Rank Adaptation) — PEFT

Fine-tune <0.1% of parameters by injecting low-rank updates

$W' = W + \text{delta}_W = W + B \cdot A$ where B in $\mathbb{R}^{(d \times r)}$, A in $\mathbb{R}^{(r \times k)}$, $r \ll \min(d, k)$
Trainable params $\sim r(d+k)$ instead of $d \cdot k$ For $r=8$, $d=k=4096$: 65,536 vs 16,777,216 params trained

Why it works: Fine-tuning updates tend to be low-rank (intrinsic dimensionality hypothesis). Learn only B and A; merge $W \leftarrow W + BA$ at inference for zero latency overhead. QLoRA adds 4-bit quantization for more memory savings.

G. RLHF & Alignment

G1 — Reward Model Training (Bradley-Terry)

Train on human pairwise preferences

$$L_{RM} = -\log \sigma(r_{\theta}(x, y_w) - r_{\theta}(x, y_l)) \quad P(y_w > y_l | x) = \frac{\sigma(r_w - r_l)}{1 + \exp(-(r_w - r_l))}$$

Symbol	Plain English
y_w (winner)	The response a human preferred
y_l (loser)	The response a human did NOT prefer
$r_{\theta}(x, y)$	Reward model's scalar score for response y given prompt x
$\sigma(r_w - r_l)$	Probability of preferring y_w . We MAXIMIZE this during training.

G2 — RLHF Regularized Objective (KEY FORMULA)

What we MAXIMIZE during PPO fine-tuning

$$J(\theta) = E_{\pi_{\theta}(y|x)} [r(x, y) - \lambda \log(\pi_{\theta}(y|x) / \pi_{ref}(y|x))] = E[\text{reward}] - \lambda \text{KL}(\pi_{\theta} || \pi_{ref})$$

Symbol	Meaning	Plain English
π_{θ}	Current policy (LLM being trained)	The model we are improving
π_{ref}	Reference policy (frozen SFT model)	The original fine-tuned model — our 'anchor'
$r(x, y)$	Reward from reward model	How good is this response according to human preferences?
$\text{KL}(\pi \pi_{ref})$	KL divergence from reference	How different has our model become from the starting point?
λ (beta)	Regularisation strength	High = stay close to reference; Low = optimize reward aggressively

Why the KL term exists: Without it, the model 'hacks' the reward model — produces gibberish that scores high (e.g., repeating praise words). KL penalizes deviation from the SFT baseline, keeping outputs fluent and diverse. KL is vs π_{ref} , NOT vs the reward model.

G3 — PPO-CLIP Surrogate Objective

Prevents catastrophic policy updates via trust-region clipping

$$L^{CLIP}(\theta) = E[\min(rt \cdot \hat{A}, \text{clip}(rt, 1-\epsilon, 1+\epsilon) \cdot \hat{A})] \quad rt = \frac{\pi_{\theta}(a|s)}{\pi_{old}(a|s)} \leftarrow \text{probability ratio}$$

Symbol	Plain English
rt	Ratio of new to old policy probability. $rt=1$ means no change.
$\hat{A}$	Advantage: was this action better (+) or worse (-) than average?
$\text{clip}(rt, 1-\epsilon, 1+\epsilon)$	Cap ratio within [0.8, 1.2] if $\epsilon=0.2$ -> no gradient outside this range
$\min(.)$	Pessimistic bound -> updates outside trust region receive no gradient signal

G4 — RL Framework for LLMs

RL Concept	LLM Equivalent	Plain English
State s_t	Prompt x + all generated tokens $y_{<t}$	Everything the model has seen and produced so far
Action a_t	Next generated token	The model's 'move' — choosing one word from vocabulary
Policy $\pi(a s)$	LLM's softmax distribution over vocabulary	Probability the model assigns to each possible next token
Reward	$r(x,y)$ at sequence end	Human preference score — given only at the end (sparse)
Environment	Text input + generated output + feedback	The whole system the agent (LLM) interacts with

H. CNN: Dropout, Batch Normalization & Attention

H1 — Batch Normalization

Normalize activations within each mini-batch

```
mu_B = (1/m) sum xi          <- batch mean
sigma2_B = (1/m) sum (xi-mu_B)^2
<- batch variance
x_hat_i = (xi - mu_B) / sqrt(sigma2_B + eps)  <- standardize yi
= gamma*x_hat_i + beta          <- learnable scale and shift
```

Benefits: (1) Allows 5-10x higher learning rates (smoother loss landscape). (2) Acts as mild regularizer (noisy batch statistics inject stochasticity). (3) Less sensitive to weight initialization. At inference: use running EMA of μ and σ^2 — NOT batch statistics.

H2 — SE Block (Squeeze-and-Excitation / Channel Attention)

Per-channel scaling based on global context

```
1. Squeeze: zc = (1/HW) sum_{i,j} F[i,j,c]   (global avg pool -> scalar per channel)
2. Excite: s = sigma(W2.ReLU(W1.z))          (bottleneck FC: C -> C/r -> C)
3. Scale: F'[i,j,c] = sc * F[i,j,c]          (rescale feature map)
```

CBAM: Channel Attention (SE block) + Spatial Attention applied in series. Consistently outperforms SE alone on classification and detection tasks.

H3 — Dropout Variants for CNNs

Variant	What It Drops	When to Use
Standard Dropout	Individual activations independently	Dense/FC layers — NOT effective in conv layers
SpatialDropout	Entire feature-map channels	CNN networks — prevents channel co-adaptation
DropBlock	Contiguous spatial blocks (e.g., 7x7 patches)	Large CNNs on natural images — removes semantic regions

Standard dropout is WEAK in CNNs: Adjacent conv activations are spatially correlated — a dropped pixel can be inferred from its neighbors. Use SpatialDropout or DropBlock instead.

I. Image Segmentation

Architecture	Key Idea	Exam Points
FCN	Replace FC layers with 1x1 convs + upsample; skip connections	First fully-convolutional segmentation model
U-Net	Symmetric encoder-decoder with skip connections at every scale	Designed for tiny biomedical datasets; concat skips preserve boundaries
DeepLab	Atrous (dilated) convolutions + ASPP multi-scale context	Dilation = larger RF without losing resolution or adding parameters

Architecture	Key Idea	Exam Points
Mask R-CNN	Faster RCNN + mask head; RoIAlign replaces RoIPool	RoIAlign uses bilinear interpolation — eliminates quantization error for pixel masks

Key Segmentation Losses

Dice Loss: $L_{Dice} = 1 - \frac{2|A \cap B|}{(|A| + |B|)}$ <- optimizes overlap
 directly Focal Loss: $L_{Focal} = -\alpha \cdot (1 - p_t)^\gamma \cdot \log(p_t)$ <- down-weights easy
 examples mIoU: $IoU_c = \frac{TP_c}{TP_c + FP_c + FN_c}$ <- standard
 evaluation metric

J. Agentic Frameworks & RAG

Framework	Core Idea	Key Distinguisher
ReAct	Interleave Thought -> Action -> Observation each step	External tool calls + reasoning together; $O(T^2)$ LLM calls
Reflexion	After each trial, generate verbal reflection stored in memory	Improves across trials WITHOUT weight updates; cross-trial memory
ReWOO	Plan ALL tool calls upfront, execute in batch, then solve	Decouples planning from execution -> fewer LLM calls; more efficient
Self-Refine	Generate -> Feedback -> Refine loop within single trial	Single-trial improvement; feedback stage identifies specific weaknesses
HuggingGPT	4 stages: Task Planning -> Model Selection -> Execution -> Response	LLM as controller orchestrating Hugging Face models

J1 — RAG Variants

Model	Generation Approach	Best Use Case
REALM	Single document + question encoded together	Open-domain QA; single-doc grounding required
RAG-Sequence	Full answer per passage, then marginalise across passages	Legal/medical QA — each answer from one document
RAG-Token	Each OUTPUT TOKEN marginalised over all passages	When different parts of answer may come from different docs
FiD	Passages encoded separately, fused in decoder cross-attention	Best accuracy; scales better than concat ($O(KL^2)$ vs $O((KL)^2)$)

Part 2: Worked Examples

Step-by-step solutions at three difficulty levels. In an open-book exam, use these as templates for your answers.

Simple Level Examples

S1 — Bigram Probability Calculation

Q: Corpus: 'the cat sat. the dog ran. the cat ran.' Compute $P(\text{cat}|\text{the})$ using MLE bigrams. Then apply Laplace smoothing with $V=5$.

Step 1: Count the bigrams

```
Count('the', 'cat')=2, Count('the', 'dog')=1
Count('the')=3, Vocabulary V=5
```

Step 2: MLE estimate

```
P_MLE(cat|the) = Count(the, cat) / Count(the)
                = 2/3 ≈ 0.667
```

Step 3: Laplace smoothed estimate

```
P_Laplace(cat|the) = (Count(the, cat)+1) / (Count(the)+V)
                   = (2+1) / (3+5) = 3/8 = 0.375
```

Answer: MLE = 0.667 | Laplace = 0.375. Smoothing reduced probability to give mass to unseen bigrams.

S2 — Perplexity Calculation

Q: A language model assigns probabilities 0.5, 0.25, 0.125 to a 3-token test sequence. Compute perplexity.

Step 1: Take natural log of each probability

```
log(0.5)    = -0.693
log(0.25)   = -1.386
log(0.125)  = -2.079
```

Step 2: Sum and average

```
Sum = -0.693 + (-1.386) + (-2.079) = -4.158
Average = -4.158 / 3 = -1.386 (this is -H, negative cross-entropy)
```

Step 3: Apply perplexity formula

```
PP = exp(-average) = exp(1.386) ≈ 4.0
```

Answer: PP = 4.0. The model is as confused as a uniform 4-way choice. Geometric mean of {0.5, 0.25, 0.125} = 1/4.

S3 — LSTM Gate Numerical Calculation

Q: In an LSTM at time t : $f_t=0.9$, $i_t=0.4$, $\hat{c}_t=0.8$, $o_t=0.7$, $C_{t-1}=1.2$. Compute C_t and h_t .

Step 1: Compute new cell state

```
C_t = f_t * C_{t-1} + i_t * \hat{c}_t
     = 0.9 * 1.2 + 0.4 * 0.8
     = 1.08 + 0.32
     = 1.40
```

Step 2: Compute hidden state

```
h_t = o_t * tanh(C_t)
     = 0.7 * tanh(1.40)
     = 0.7 * 0.885 [tanh(1.4) ≈ 0.885]
     ≈ 0.620
```

Answer: $C_t = 1.40$, $h_t \approx 0.620$. Forget gate (0.9) kept 90% of old memory. Input gate (0.4) added 40% of new candidate.

S4 — Bradley-Terry Reward Probability

Q: Reward model gives $r(x, y_w) = 2.5$, $r(x, y_l) = 1.0$. Compute $P(y_w \text{ preferred over } y_l)$.

Step 1: Apply Bradley-Terry formula

$$P(y_w > y_l) = \text{sigma}(r_w - r_l) = \text{sigma}(2.5 - 1.0) = \text{sigma}(1.5)$$

Step 2: Compute sigmoid

$$\text{sigma}(1.5) = 1 / (1 + \exp(-1.5)) = 1 / (1 + 0.223) = 1 / 1.223 \approx 0.818$$

Answer: $P(y_w \text{ preferred}) \approx 81.8\%$. A reward gap of 1.5 gives ~82% confidence in the preference ranking.

Medium Level Examples

M1 — Scaled Self-Attention Numerical (Full Walkthrough)

Q: $T=3$, $d_k=2$. $Q=[[1,0],[0,1],[1,1]]$, $K=[[1,0],[0,1],[1,1]]$, $V=[[1,0],[0,1],[1,1]]$. Compute attention output for position 1.

Step 1: Compute raw scores for row 1 ($q_1=[1,0]$)

$$\begin{aligned} e_{11} &= q_1 \cdot k_1 = [1,0] \cdot [1,0] = 1 \\ e_{12} &= q_1 \cdot k_2 = [1,0] \cdot [0,1] = 0 \\ e_{13} &= q_1 \cdot k_3 = [1,0] \cdot [1,1] = 1 \end{aligned}$$

Step 2: Scale by $\text{sqrt}(d_k) = \text{sqrt}(2) \approx 1.414$

$$\text{scaled} = [1/1.414, 0/1.414, 1/1.414] = [0.707, 0, 0.707]$$

Step 3: Apply softmax

$$\begin{aligned} \exp([0.707, 0, 0.707]) &= [2.028, 1.0, 2.028]; \quad \text{sum} = 5.056 \\ \alpha &= [2.028/5.056, 1.0/5.056, 2.028/5.056] = [0.401, 0.198, 0.401] \end{aligned}$$

Step 4: Compute weighted sum of values

$$\begin{aligned} a_1 &= 0.401 \cdot [1,0] + 0.198 \cdot [0,1] + 0.401 \cdot [1,1] \\ &= [0.401, 0] + [0, 0.198] + [0.401, 0.401] \\ &= [0.802, 0.599] \end{aligned}$$

Answer: Output at position 1 = $[0.802, 0.599]$. Attends equally to positions 1 and 3 (both match $q_1=[1,0]$), less to position 2.

M2 — Batch Normalization Forward Pass

Q: Mini-batch for one channel: $x=[2,4,4,4,8]$. Compute BN output with $\gamma=1$, $\beta=0$, $\text{eps}=1e-5$.

Step 1: Compute batch mean

$$\mu_B = (2+4+4+4+8)/5 = 22/5 = 4.4$$

Step 2: Compute batch variance

$$\begin{aligned} \text{sigma2}_B &= [(2-4.4)^2 + (4-4.4)^2 \cdot 3 + (8-4.4)^2] / 5 \\ &= [5.76 + 0.48 + 12.96] / 5 = 19.2/5 = 3.84 \end{aligned}$$

Step 3: Standardize each activation

$$\begin{aligned} x_{\text{hat}_i} &= (x_i - 4.4) / \text{sqrt}(3.84) = (x_i - 4.4) / 1.9596 \\ x_{\text{hat}_1} &= (2-4.4)/1.9596 \approx -1.225 \\ x_{\text{hat}_2} &= (4-4.4)/1.9596 \approx -0.204 \quad [x_3=x_4 \text{ same}] \\ x_{\text{hat}_5} &= (8-4.4)/1.9596 \approx +1.837 \end{aligned}$$

Step 4: Apply scale/shift ($\gamma=1$, $\beta=0 \rightarrow$ identity)

$$\text{Output} \approx [-1.225, -0.204, -0.204, -0.204, +1.837]$$

Answer: Output is zero-mean, unit-variance. The outlier (8) is pulled in; the low value (2) is pushed proportionally negative.

M3 — Bahdanau Attention Worked Example

Q: Encoder states: $h_1=[4,0]$, $h_2=[0,3]$, $h_3=[2,2]$. Decoder step 1 state: $s_0=[1,1]$. Using dot-product scoring, compute alpha and context vector c_1 .

Step 1: Compute alignment scores $e_{1j} = s_0 \cdot h_j$

$$\begin{aligned} e_{11} &= [1,1] \cdot [4,0] = 4 \\ e_{12} &= [1,1] \cdot [0,3] = 3 \\ e_{13} &= [1,1] \cdot [2,2] = 4 \end{aligned}$$

Step 2: Softmax over scores

$$\begin{aligned} \exp([4,3,4]) &= [54.60, 20.09, 54.60]; \quad \text{sum} = 129.29 \\ \alpha &= [54.60/129.29, 20.09/129.29, 54.60/129.29] \\ &\approx [0.422, 0.155, 0.422] \end{aligned}$$

Step 3: Compute context vector $c_1 = \sum \alpha_j h_j$

$$\begin{aligned} c_1 &= 0.422 \cdot [4,0] + 0.155 \cdot [0,3] + 0.422 \cdot [2,2] \\ &= [1.688, 0] + [0, 0.465] + [0.844, 0.844] \\ &= [2.532, 1.309] \end{aligned}$$

Answer: Attention split equally between h_1 and h_3 (both score 4), less on h_2 . Context vector is a weighted blend representing 'what to look at'.

Complex Level Examples

C1 — BERT-base Parameter Count (Full Derivation)

Q: BERT-base: $d=768$, $h=12$ heads, $L=12$ layers, FFN expansion=4x, vocab=30,522. Estimate total parameters.

Step 1: Embedding layer

$$\begin{aligned} \text{Token embeddings: } & 30,522 \times 768 = 23,440,896 \\ \text{Position embeddings: } & 512 \times 768 = 393,216 \\ \text{Segment embeddings: } & 2 \times 768 = 1,536 \\ \text{Total embedding } & \approx 23.8\text{M} \end{aligned}$$

Step 2: One transformer layer — MHA

$$\begin{aligned} W_Q, W_K, W_V: & \text{each } 768 \times 768 = 589,824 \text{ params} \\ W_O: & 768 \times 768 = 589,824 \text{ params} \\ \text{Total MHA} &= 4 \times 768^2 = 4 \times 589,824 \approx 2.36\text{M per layer} \end{aligned}$$

Step 3: One transformer layer — FFN

$$\begin{aligned} W_1: & 768 \times 3072 = 2,359,296 \text{ params} \\ W_2: & 3072 \times 768 = 2,359,296 \text{ params} \\ \text{Total FFN} &= 2 \times 768 \times 3072 \approx 4.72\text{M per layer} \end{aligned}$$

Step 4: Total across all layers + embeddings

$$\begin{aligned} \text{Per layer } &\approx 2.36\text{M} + 4.72\text{M} = 7.08\text{M (excluding LN/biases)} \\ 12 \text{ layers: } &12 \times 7.08\text{M} = 84.96\text{M} \\ + \text{ Embeddings: } &23.8\text{M} \\ + \text{ Pooler: } &0.59\text{M} \\ \text{Total } &\approx 110\text{M parameters} \quad \leftarrow \text{ matches known BERT-base size} \end{aligned}$$

Answer: ~110M parameters. Key ratio: MHA $\approx 4d^2$, FFN $\approx 8d^2$, so FFN holds ~2x more params than MHA per layer.

C2 — RLHF Optimal Policy Derivation

Q: Show that the closed-form optimal policy for $J(\theta) = E[r(x,y)] - \beta \text{KL}(\pi_\theta || \pi_{\text{ref}})$ is $\pi(y|x)$ proportional to $\pi_{\text{ref}}(y|x) \cdot \exp(r(x,y)/\beta)$.

Step 1: Expand the KL divergence

$$\text{KL}(\pi_\theta || \pi_{\text{ref}}) = \sum_y \pi_\theta(y|x) \cdot \log[\pi_\theta(y|x) / \pi_{\text{ref}}(y|x)]$$

Step 2: Write out full objective

```
J(theta) = sum_y pi_theta(y|x) [r(x,y) - beta*log(pi_theta(y|x)/pi_ref(y|x))]
```

Step 3: Take functional derivative w.r.t. $\pi(y|x)$ and set to zero (with Lagrange multiplier)

```
d/d_pi(y): r(x,y) - beta*log(pi(y|x)/pi_ref(y|x)) - beta = lambda
=> log(pi(y|x)/pi_ref(y|x)) = r(x,y)/beta - 1 - lambda/beta
=> pi*(y|x) = pi_ref(y|x) * exp(r(x,y)/beta) / Z(x)
```

Step 4: Normalize (Z is the partition function)

```
Z(x) = sum_y pi_ref(y|x)*exp(r(x,y)/beta)
Therefore: pi*(y|x) proportional to pi_ref(y|x)*exp(r(x,y)/beta)
```

Answer: The optimal policy up-weights responses that both have high prior probability AND high reward. Beta controls the trade-off. This is the basis of DPO (Direct Preference Optimization).

C3 — PPO-CLIP: Gradient Behavior Analysis

Q: At timestep t: $A_t = +2.0$ (good action), $r_t = \pi_\theta / \pi_{old} = 1.35$. With $\epsilon = 0.2$, compute L^{CLIP} and explain gradient behavior.

Step 1: Compute unclipped surrogate

```
Unclipped = r_t * A_t_hat = 1.35 * 2.0 = 2.70
```

Step 2: Compute clipped surrogate

```
clip(r_t, 1-0.2, 1+0.2) = clip(1.35, 0.8, 1.2) = 1.2    <- capped!
Clipped = 1.2 * 2.0 = 2.40
```

Step 3: Apply min (pessimistic bound)

```
L^CLIP = min(2.70, 2.40) = 2.40
```

Step 4: Gradient analysis

```
L^CLIP = 2.40 = const * A_t_hat (independent of r_t after clipping)
dL^CLIP/d_theta = 0 w.r.t. the ratio r_t
-> No gradient signal for updating the policy beyond the trust region
-> Optimizer cannot make r_t even larger (no incentive -> stable training)
```

Answer: When $r_t > 1 + \epsilon$ for a good action, clipped objective removes the gradient — policy cannot over-commit to this action. This is PPO's core safety mechanism.

Deep Learning & Generative AI — Final Examination

Centre for Machine Intelligence and Data Science

Duration	Total Marks	Format	Date
3 Hours	50 Marks	Open Book	End Semester May 2026

Instructions: (1) For MCQ/True-False, choose the correct option with a ≤ 3 -line justification. (2) Begin each section on a new page. (3) All standard formulae may be used from notes. (4) Suggested: Section 1 = 35 min, Sections 2–11 = 145 min.

Section 1 — Short Answers / MCQ / True-False [20 × 1 = 20 Marks]

Q 1.1 [1 mark]

True/False: GloVe uses negative sampling to learn word vectors.

Model Answer:

False. GloVe directly minimises weighted squared error on log co-occurrence counts using global matrix statistics. It does NOT use negative sampling — that is word2vec's technique (SGNS). This is a frequent exam trap.

Q 1.2 [1 mark]

The output vector at position l in self-attention is: A. $a_l = \sum \alpha_{lt} * q_t$ B. $a_l = \sum \alpha_{lt} * k_t$ C. $a_l = \sum \alpha_{lt} * v_t$ [CORRECT] D. $a_l = \max(\alpha_{lt} * v_t)$

Model Answer:

C. The output is a weighted sum of VALUE vectors. Attention weights α are computed from $Q.K^T$, but the aggregated content comes from V .

Q 1.3 [1 mark]

True/False: In LSTM, when $f_t \approx 1$ and $i_t \approx 0$, the cell state $C_t \approx C_{t-1}$.

Model Answer:

True. $C_t = f_t * C_{t-1} + i_t * c_t_{\text{hat}} \approx 1 * C_{t-1} + 0 * c_t_{\text{hat}} = C_{t-1}$. This is the 'constant error carousel' enabling long-range memory.

Q 1.4 [1 mark]

Which pre-training objective uses span corruption with sentinel tokens (`<extra_id_0>`)? A. BERT (MLM) B. GPT (Causal LM) C. T5 [CORRECT] D. BART

Model Answer:

C — T5. T5 replaces contiguous spans with single sentinel tokens. BERT masks individual tokens. BART uses multiple noise types. GPT uses causal LM.

Q 1.5 [1 mark]

True/False: Increasing beam size in beam search yields sequences of smaller probability.

Model Answer:

False. Larger beam is monotonically non-worse — it considers a superset of candidates. The returned sequence is at least as probable as with a smaller beam.

Q 1.6 [1 mark]

In which RAG variant does each output token marginalise over all retrieved passages independently? A. REALM B. RAG-Sequence C. RAG-Token [CORRECT] D. FiD

Model Answer:

C — RAG-Token. Each output token can be sourced from different passages. RAG-Sequence generates entire answer from one passage at a time, then marginalises over full answers.

Q 1.7 [1 mark]

True/False: In Bahdanau attention, there is a strict one-to-one mapping between encoder and decoder tokens.

Model Answer:

False. Bahdanau is many-to-many (soft alignment). Multiple decoder steps may focus on the same encoder position. No one-to-one constraint.

Q 1.8 [1 mark]

In RLHF, the KL regularisation term penalises divergence from: A. The reward model B. The reference SFT policy π_{ref} [CORRECT] C. Human annotators D. Uniform prior

Model Answer:

B. $\text{KL}(\pi_{\theta} || \pi_{\text{ref}})$ where π_{ref} is the frozen SFT model. This prevents reward hacking while keeping the model close to the SFT baseline.

Q 1.9 [1 mark]

Which positional encoding makes attention dot product $q_i \cdot k_j$ depend only on relative offset $i-j$? A. Sinusoidal B. Learned Absolute (BERT) C. RoPE [CORRECT] D. ALiBi

Model Answer:

C — RoPE. Rotary PE rotates (q, k) by an angle proportional to position. The inner product then depends only on $i-j$. ALiBi adds a linear bias — a different mechanism.

Q 1.10 [1 mark]

True/False: RAG makes LLMs more prone to hallucination compared to closed-book generation.

Model Answer:

False. RAG reduces hallucination by grounding generation in retrieved documents. Users can inspect which passages led to any answer and fix the retriever if wrong.

Q 1.11 [1 mark]

Which agentic framework improves across trials using verbal self-reflection stored in memory, WITHOUT updating weights? A. ReAct B. ReWOO C. Reflexion [CORRECT] D. Self-Refine

Model Answer:

C — Reflexion. Stores verbal reflections across trials as episodic memory. Self-Refine only iterates within a single trial. ReWOO focuses on efficiency.

Q 1.12 [1 mark]

True/False: Standard dropout is effective at regularizing CNN convolutional layers.

Model Answer:

False. Adjacent activations in CNN feature maps are spatially correlated — dropped pixels can be inferred from neighbors. Use SpatialDropout or DropBlock instead.

Q 1.13 [1 mark]

The primary reason base LLMs fail at zero-shot instruction following is: A. Insufficient parameters B. Pre-training data not in instruction format [CORRECT] C. Lack of RLHF D. Cannot process system prompts

Model Answer:

B. Base models are trained on raw text (next-token prediction). They haven't seen instruction-response format, so they continue text rather than following commands. SFT fixes this.

Q 1.14 [1 mark]

Which HuggingGPT stage parses the user request into structured sub-tasks with dependencies? A. Model Selection B. Task Planning [CORRECT] C. Task Execution D. Response Generation

Model Answer:

B — Task Planning. The LLM controller converts the user query into a DAG of sub-tasks. Model Selection then picks which Hugging Face model handles each sub-task.

Q 1.15 [1 mark]

True/False: In PPO-CLIP, when advantage is positive and $rt > 1 + \epsilon$, gradient remains proportional to $rt \cdot \hat{A}_t$.

Model Answer:

False. When $\hat{A}_t > 0$ and $rt > 1 + \epsilon$, the surrogate is capped at $(1 + \epsilon) \cdot \hat{A}_t$ — a constant w.r.t. θ . No gradient flows. This prevents over-updating on already-good actions.

Q 1.16 [1 mark]

Which CNN block computes per-channel scaling via: Global-Avg-Pool $\rightarrow$ Bottleneck-FC $\rightarrow$ Sigmoid? A. Spatial Attention B. SE (Squeeze-and-Excitation) [CORRECT] C. CBAM D. DropBlock

Model Answer:

B — SE block. Squeeze: global average pool $\rightarrow$ scalar per channel. Excite: bottleneck FC, then sigmoid $\rightarrow$ per-channel weights in (0,1). Scale: multiply feature maps.

Q 1.17 [1 mark]

Mask R-CNN replaces RoIPool with RoIAlign primarily to: A. Speed up training B. Eliminate spatial quantization that harms pixel masks [CORRECT] C. Allow variable batch sizes D. Enable multi-scale features

Model Answer:

B. RoIPool snaps to integer pixel coordinates twice, introducing ~ 0.5 -pixel misalignment — fine for boxes, catastrophic for pixel masks. RoIAlign uses bilinear interpolation.

Q 1.18 [1 mark]

True/False: A decoder-only model can follow positional order even without positional encoding.

Model Answer:

True. With a causal mask, position l can only attend to positions $\leq l$. The count of available positions is itself implicit positional information.

Q 1.19 [1 mark]

Which agentic framework explicitly decouples reasoning/planning from tool execution to reduce LLM calls? A. ReAct B. Reflexion C. ReWOO [CORRECT] D. Self-Refine

Model Answer:

C — ReWOO. One LLM call produces a full plan with tool placeholders $\rightarrow$ worker executes all tools in batch $\rightarrow$ solver integrates results. Avoids ReAct's per-step re-reading of history.

Q 1.20 [1 mark]

For machine translation (English $\rightarrow$ French), which architecture is most appropriate? A. Encoder-only (BERT) B. Decoder-only (GPT) C. Encoder-Decoder D. Decoder or Encoder-Decoder [CORRECT]

Model Answer:

D. Translation is seq2seq — naturally suited to Encoder-Decoder (T5/BART). Modern large decoder-only models (with instruction prompting) also perform well. Encoder-only cannot generate.

Section 2 — CNN Regularisation [5 Marks]

Q 2.1 [2 marks]

Describe Dropout as a regularization technique. Explain: (a) training vs inference behavior, (b) why standard dropout is weak in CNNs, and (c) name two better CNN alternatives with justification.

Model Answer:

(a) Training: Each forward pass draws mask $m_i \sim \text{Bernoulli}(1-p)$ per activation. Active neurons are scaled by $1/(1-p)$ (inverted dropout), keeping expected output unchanged. Inference: No dropout — use all neurons as-is.

(b) Weakness in CNNs: Adjacent activations in a feature map are spatially correlated (same filter output at nearby pixels). Dropping one pixel leaves its neighbors intact, which easily reconstruct the dropped value — removing little information.

(c) Better alternatives:

- SpatialDropout: Drops entire feature-map channels. Forces the network to learn without any activation from that filter, preventing channel co-adaptation.
- DropBlock: Drops contiguous spatial blocks (e.g., 7x7 patches). Removes semantically meaningful regions so neighbors cannot compensate.

Q 2.2 [3 marks]

Write the Batch Normalization transform in full. Explain: (a) why it allows higher learning rates, (b) how it acts as a regularizer, and (c) what changes at inference time.

Model Answer:

Transform: $\mu_B = (1/m) \sum(x_i)$ | $\sigma_B^2 = (1/m) \sum((x_i - \mu_B)^2)$ | $\hat{x}_i = (x_i - \mu_B) / \sqrt{\sigma_B^2 + \epsilon}$ | $y_i = \gamma \hat{x}_i + \beta$

(a) Higher LRs: BN normalizes activations, smoothing the loss landscape and bounding gradient norms. Gradients propagate to deep layers without saturating or exploding, allowing 5-10x larger learning rates.

(b) Regularization: μ_B and σ_B^2 are computed over each mini-batch — they are noisy estimates. This noise injects stochasticity into activations during training (similar to dropout), reducing reliance on any single neuron.

(c) Inference: Cannot use batch statistics with 1 sample. Use exponential moving averages μ_{running} and σ_{running} accumulated during training: $\hat{x} = (x - \mu_{\text{running}}) / \sqrt{\sigma_{\text{running}}^2 + \epsilon}$, then apply learned γ , β .

Section 3 — Attention [5 Marks]

Q 3.1 [1 mark]

Define attention in one sentence. Give one NLP and one computer vision example.

Model Answer:

Attention is a mechanism that computes a weighted combination of input elements (values) where weights measure relevance to a query.

NLP: In English->French translation, when generating 'chat' (cat), the decoder attends heavily to the encoder's 'cat' token and nearly ignores other words.

CV: In image captioning, when generating 'ball', spatial attention highlights the image region containing the ball, suppressing background pixels.

Q 3.2 [2 marks]

Distinguish spatial attention from channel/spectral attention. Give one application domain for each.

Model Answer:

Spatial Attention: Assigns importance weights over the $H \times W$ spatial locations of a feature map. Multiplies a 2D attention map by the feature map to emphasize relevant regions. Application: medical image analysis (attending to a small lesion while ignoring normal tissue).

Channel/Spectral Attention (SE block): Assigns importance weights over the C feature channels (or spectral bands). Rescales each channel by a learned scalar. Application: hyperspectral remote sensing — up-weight NIR bands for vegetation, down-weight noisy visible bands.

Q 3.3 [2 marks]

Write the scaled dot-product self-attention equation and explain why we divide by $\sqrt{d_k}$.

Model Answer:

Equation: $\text{Attention}(Q, K, V) = \text{softmax}(QK^T / \sqrt{d_k}) * V$

Why $\sqrt{d_k}$: Each entry of $Q \cdot K^T$ is a dot product of d_k -dimensional vectors. If entries are i.i.d. zero-mean unit-variance, the dot product has variance d_k . As d_k grows, dot products become very large $\rightarrow$ very peaked softmax $\rightarrow$ near-zero gradients for all non-max entries $\rightarrow$ learning stalls. Dividing by $\sqrt{d_k}$ keeps pre-softmax variance ≈ 1 regardless of d_k .

Section 4 — Transformer Details [6 Marks]

Q 4.1 [2 marks]

Explain the motivation for Multi-Head Attention over a single attention head. Write the equation combining heads into the final output.

Model Answer:

Motivation: A single softmax attention head tends to be peaky — it focuses all weight on one dominant relationship and cannot simultaneously attend to multiple dependency types. Multi-head attention runs h independent attention computations in parallel, each in a d/h -dimensional subspace, letting each head specialize.

Equation:

$\text{head}_i = \text{Attention}(X \cdot W_{Q_i}, X \cdot W_{K_i}, X \cdot W_{V_i})$

$\text{MHA}(X) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) \cdot W_O$

Each $W_{Q_i}, W_{K_i}, W_{V_i}$ in $\mathbb{R}^{(d \times d/h)}$. W_O in $\mathbb{R}^{(d \times d)}$ mixes concatenated outputs.

Q 4.2 [2 marks]

Why is causal masking required in the decoder during training? Give the mathematical form and state the property it preserves.

Model Answer:

Why: During training, the entire target sequence is fed at once (teacher forcing) for efficiency. Without masking, position l could see all future positions $l+1, l+2, \dots$ — effectively cheating. At inference these future tokens do not exist, making training and inference inconsistent.

Mathematical form: Set $e_{\{l,t\}} = -\infty$ for all $t > l$ (before softmax). After softmax, $\alpha_{\{l,t\}} = 0$ for $t > l$.

Property preserved: Auto-regressive property — prediction at position l depends only on positions $1, \dots, l$ (past and present), consistent with left-to-right generation at inference.

Q 4.3 [2 marks]

Compare sinusoidal, learned absolute, and RoPE positional encodings. State one strength and one weakness of each.

PE Type	Strength	Weakness
Sinusoidal	No parameters; extrapolates to longer sequences in principle	Fixed; only absolute position — not inherently relative
Learned Absolute (BERT/GPT-2)	Flexible; optimised end-to-end for the task	Cannot extrapolate beyond training length
RoPE (LLaMA)	Dot product $q_i \cdot k_j$ depends only on $i-j$ (true relative); moderate extrapolation	Slightly more compute; some saturation at very long offsets

Section 5 — Pre-training & Fine-tuning [4 Marks]

Q 5.1 [2 marks]

Compare BERT's MLM with GPT's Causal LM objective. Why is MLM unsuitable for free-form text generation?

Model Answer:

MLM (BERT): Randomly mask $\sim 15\%$ of tokens; train to predict them using bidirectional context (both left and right neighbors). Rich bidirectional representations. Never learns full left-to-right generation.

Causal LM (GPT): Predict the next token using only left context (causal mask enforced). Naturally supports auto-regressive text generation.

Why MLM fails for generation: (1) At inference, all right-context tokens are unknown — BERT was trained to use both sides. (2) BERT fills individual gaps, not full sequences; it has no notion of what comes after the generated token. (3) Standard decoding strategies (beam search, sampling) are designed for next-token prediction — incompatible with MLM gap-filling.

Q 5.2 [2 marks]

A practitioner wants to fine-tune a 7B LLM for legal document summarization on a single 24GB GPU. Recommend a PEFT method, write the mathematical update, and justify your choice.

Model Answer:

Recommendation: LoRA (Low-Rank Adaptation), or QLoRA if memory is very tight.

Math: Instead of updating full weight W in $\mathbb{R}^{(d \times k)}$, inject: $W' = W + \Delta W = W + B \cdot A$, where B in $\mathbb{R}^{(d \times r)}$, A in $\mathbb{R}^{(r \times k)}$, with $r \ll \min(d, k)$ (typically $r=8-16$). Only B and A are trained.

Justification: Trainable parameters $\sim r(d+k) \sim 0.01-0.1\%$ of original model — easily fits in 24GB. Fine-tuning updates tend to be low-rank in practice (intrinsic dimensionality hypothesis), so expressiveness loss is minimal. At inference: merge $W \leftarrow W + BA$, zero extra latency.

Section 6 — Prompting & In-Context Learning [2 Marks]**Q 6.1** [2 marks]

Explain why Chain-of-Thought (CoT) prompting improves multi-step reasoning. State two conditions under which CoT fails to help.

Model Answer:

Why CoT works: CoT provides intermediate reasoning steps in few-shot examples or via 'Let's think step by step.' The model generates explicit intermediate computations before the final answer, using extra generated tokens as a 'scratch pad.' This decomposes hard problems into simpler sub-problems. Empirically, large benefit emerges only in models $\geq \sim 60B$ parameters.

Two conditions where CoT fails:

- (1) Single-step tasks (e.g., sentiment classification) — no multi-step reasoning chain to exploit; CoT adds tokens with no benefit.
- (2) Small models (below $\sim 7B$) — cannot generate coherent reasoning chains; the 'thinking steps' are incoherent and propagate errors rather than helping.

Section 7 — Instruction Tuning & Alignment [4 Marks]**Q 7.1** [2 marks]

Why is SFT (Supervised Fine-Tuning) alone insufficient for producing safe and aligned LLMs? Give a concrete failure example.

Model Answer:

Why insufficient: SFT trains the model to maximise likelihood of (prompt, response) pairs — it learns to be fluent and on-topic but has no mechanism to prefer safe over unsafe responses. SFT provides a signal about what responses look like, not which responses are better when multiple completions are possible.

Failure example: Prompt: 'How can I hack into my neighbor's WiFi?' An SFT model trained on helpful Q&A might generate step-by-step instructions (because 'being helpful' is the only signal). An RLHF-aligned model would refuse, because human preference data explicitly ranked refusals higher than harmful guides. RLHF expresses that some responses are better — SFT cannot.

Q 7.2 [2 marks]

Describe the Evol-Instruct method for generating synthetic instruction-tuning data. Name at least three evolution operations the Instruction Evolver applies.

Model Answer:

Evol-Instruct starts with a seed set of simple instructions, then iteratively complicates them using an 'Instruction Evolver' LLM. Each generation produces harder instructions; failures (too hard, off-topic, invalid) are filtered out.

Evolution operations:

- (1) Add Constraints — impose word limits, format requirements, or banned words
- (2) Deepen — require greater depth, expertise, or specificity in the response
- (3) Concretize — replace vague terms with specific instances
- (4) Increase reasoning steps — require more logical deduction steps
- (5) Complicate input — add noise or ambiguity to the input data

Section 8 — RLHF Mathematics [5 Marks]

Q 8.1 [2 marks]

In the LLM-RL framework, define State (st), Action (at), and Policy (pi). What is the reward structure?

Model Answer:

State st: The sequence containing all prompt tokens x concatenated with all previously generated output tokens $y_{<t}$. The full context the model can see at step t.

Action at: The single output token generated at timestep t, drawn from vocabulary V. The 'move' the LLM makes at each step.

Policy $\pi_{\theta}(a|st)$: The LLM's probability distribution over vocabulary V given state st, parameterised by weights theta. This is literally the softmax output of the language model.

Reward: Sparse — reward $r(x,y)$ is given only at the end of the full sequence (final token), not at each token. This is what makes RLHF challenging — credit must be assigned back through the entire sequence.

Q 8.2 [3 marks]

Write the complete regularised reward-maximisation objective used in RLHF. Explain the role of each term and what happens if KL coefficient beta is set to zero.

Model Answer:

Objective:

$$\max_{\theta} E_{x \sim D, y \sim \pi_{\theta}(.|x)} [r_{\phi}(x,y) - \beta \log(\pi_{\theta}(y|x)/\pi_{\text{ref}}(y|x))] \\ = E[\text{reward}] - \beta * KL(\pi_{\theta}(.|x) || \pi_{\text{ref}}(.|x))$$

Term-by-term:

- $r_{\phi}(x,y)$: Reward from the pre-trained reward model (Bradley-Terry). Proxy for human preference. We maximize this.
- $KL(\pi_{\theta}(.|x) || \pi_{\text{ref}})$: How far our trained policy has drifted from the SFT reference policy. We minimize this.
- beta: Trade-off coefficient. High beta = stay close to SFT; Low beta = aggressively optimize reward.
- π_{ref} : Frozen SFT model — the 'anchor' preventing degenerate solutions.

If beta=0: No regularisation -> policy drifts arbitrarily far from π_{ref} , producing 'reward-hacked' outputs (e.g., repeating certain high-reward phrases indefinitely). Language model distribution degenerates and output diversity collapses.

Section 9 — Algorithms (PPO) [4 Marks]

Q 9.1 [2 marks]

Explain importance sampling and how it justifies taking multiple gradient update epochs from the same rollout in PPO.

Model Answer:

Importance Sampling (IS): Estimates expectations under one distribution (π_{θ}) using samples from another (π_{old}):

$$E_{\pi_{\theta}}[f(x)] = E_{\pi_{\text{old}}}[(\pi_{\theta}(x)/\pi_{\text{old}}(x)) * f(x)]$$

The ratio $rt = \pi_{\theta}(at|st)/\pi_{\text{old}}(at|st)$ reweights each sample.

In PPO: Collect rollouts under π_{old} (the policy before the current update round). Each gradient update changes theta, making old samples slightly off-distribution. IS corrects for this by reweighting by rt. Without IS, taking multiple epochs would introduce increasing bias. With IS, samples remain valid provided ratios stay bounded — which PPO ensures via clipping (rt in $[1-\epsilon, 1+\epsilon]$). This makes multi-epoch gradient updates from a single rollout mathematically justifiable.

Q 9.2 [2 marks]

Write the PPO-CLIP surrogate objective. Explain with a numerical example ($\epsilon=0.2$, $At_{\text{hat}}=+2.0$, $rt=1.5$) how clipping prevents catastrophic policy updates.

Model Answer:

Objective: $L^{\text{CLIP}}(\theta) = E_t[\min(rt * At_{\text{hat}}, \text{clip}(rt, 1-\epsilon, 1+\epsilon) * At_{\text{hat}})]$
where $rt = \pi_{\theta}(at|st)/\pi_{\text{old}}(at|st)$

Numerical example ($\epsilon=0.2$, $At_{\text{hat}}=+2.0$, $rt=1.5$):

Unclipped: $1.5 * 2.0 = 3.0$

Clipped: $\text{clip}(1.5, 0.8, 1.2) * 2.0 = 1.2 * 2.0 = 2.4$

$L^{\text{CLIP}} = \min(3.0, 2.4) = 2.4$ <- constant w.r.t. rt

Gradient: $dL^{\text{CLIP}}/d_{\theta} = 0$ from the ratio rt beyond 1.2

The optimizer cannot push ϵ above 1+eps for a good action — no gradient signal beyond the trust region. Prevents destabilizing large policy swings.

Section 10 — Retrieval & Agents [3 Marks]

Q 10.1 [2 marks]

Differentiate RAG-Sequence from RAG-Token. For a legal document Q&A system (where mixing sources could cause contradictions), which would you choose and why?

Model Answer:

RAG-Sequence: Retrieve K passages. Generate one complete candidate answer per passage. Marginalise by taking the highest-probability full answer across passages. Each candidate answer is grounded in exactly one document.

RAG-Token: Retrieve K passages. At EACH output token, marginalise over all passages — different tokens may cite different documents. The final answer potentially mixes evidence from multiple documents per word.

Choice for legal QA: RAG-Sequence. Laws and clauses from different jurisdictions or documents can directly contradict each other. RAG-Token might mix a clause from Contract A with Contract B — producing a legally nonsensical hybrid. RAG-Sequence ensures the entire answer traces to one coherent document, maintaining traceability.

Q 10.2 [1 mark]

In one sentence, explain how ReWOO is more efficient than ReAct.

Model Answer:

ReWOO generates the entire reasoning plan with tool placeholders in a single LLM call upfront, executes all tools in batch, then integrates results in one final call — avoiding ReAct's per-step LLM call that must re-read the growing Thought/Action/Observation history each time.

Section 11 — Design Problem [6 Marks]

Q 11 [6 marks]

You are building an AI teaching assistant for an engineering university. The system must: (i) answer subject-specific questions using course materials, (ii) refuse to write exam answers for students (academic integrity), and (iii) route complex queries to human TAs when uncertain. You have a 7B open-source LLM, ~3,000 Q&A pairs from past TAs, and a vector store of course documents.

11.1 Propose a full training pipeline (data → deployed model), naming techniques at each stage. [3 marks]

11.2 Describe the inference pipeline for answering a query. [2 marks]

11.3 Which RAG variant (RAG-Sequence, RAG-Token, or FiD) would you use, and why? [1 mark]

Model Answer:

11.1 Training Pipeline:

Stage 1 — SFT (Instruction Tuning): Format the 3,000 TA Q&A pairs as (instruction, response) pairs. Train with cross-entropy loss. Augment with Self-Instruct to synthesise additional domain examples. Use LoRA ($r=16$) throughout for GPU efficiency.

Stage 2 — Reward Modeling: Collect ~5,000 pairwise preference annotations (helpful vs. unhelpful; refusal vs. answer-giving). Train a Bradley-Terry reward model: $\text{minimize } L_{\text{RM}} = -\log \sigma(r(x, y_w) - r(x, y_l))$.

Stage 3 — RLHF (PPO): Maximize $J(\theta) = E[r(x, y)] - \beta \text{KL}(\pi_{\theta} || \pi_{\text{SFT}})$. This explicitly trains the model to prefer helpful explanations over direct answer-giving, while penalizing deviation from the SFT baseline.

11.2 Inference Pipeline:

(1) Safety check: Run a lightweight classifier to detect exam-cheating requests → if detected, respond with templated refusal ('I can explain this concept, but not write your exam answer').

(2) Retrieval: Embed query using DPR encoder; retrieve top-K=5 passages from course document vector store.

(3) Generation: Condition the model on [system prompt + retrieved passages + query]. Generate response token by token.

(4) Confidence estimation: Compute length-normalised log-probability or use reward model score. If below threshold τ → route to human TA with the retrieved passages for context.

11.3 RAG Variant: FiD (Fusion-in-Decoder). Course questions often require synthesising information across multiple lecture slides or textbook chapters. FiD encodes each passage independently ($O(KL^2)$ vs $O((KL)^2)$ for concatenation) and fuses in decoder cross-attention — best accuracy for multi-document synthesis without quadratic cost.

Total: 50 Marks | End of Examination Paper

Part 4: Last-Hour Cheat Sheet

Highest-yield facts for open-book exam. If you remember nothing else, remember these.

Formula Quick Reference

Formula Name	The Formula	Key Insight
Bigram Prob	$P(w_i w_{i-1}) = C(w_{i-1}, w_i) / C(w_{i-1})$	Count pair / count context word
Laplace Smooth	$[C(w_1, w_2) + k] / [C(w_1) + k V]$	Add k to every count; prevents zeros
Perplexity	$\exp(-1/N * \sum \log P(w_i ctx))$	Lower = better; exponent of avg neg log-prob
SGNS Loss	$-\log \sigma(u_o \cdot v_c) - \sum \log \sigma(-u_n \cdot v_c)$	Push true context together, noise apart
GloVe	$\sum f(X_{ij})(w_i \cdot w_j + b_i + b_j - \log X_{ij})^2$	NO negative sampling; weighted LS on log counts
Vanilla RNN	$h_t = \tanh(Wx_t + Wh \cdot h_{t-1} + b)$	Vanishes over long sequences
LSTM Cell	$C_t = f_t * C_{t-1} + i_t * c_t$	$f_{t-1} \rightarrow$ gradient flows as identity
LSTM Hidden	$h_t = o_t * \tanh(C_t)$	Exposed portion of current cell state
GRU Final	$h_t = (1 - z_t) * h_{t-1} + z_t * h_t$	2 gates; 25% fewer params than LSTM
Self-Attention	$\text{softmax}(Q * K^T / \sqrt{d_k}) * V$	$\sqrt{d_k}$: keeps variance ~1; output = weighted sum of V
Multi-Head	$\text{Concat}(\text{head}_1, \dots, \text{head}_h) * W_O$	h heads, each $d_k = d/h$; same total budget
Sinusoidal PE	$\sin(\text{pos}/10000^{(2i/d)}), \cos(\text{pos}/10000^{(2i/d)})$	Added (not concat) to token embeddings
FFN	$\max(0, x * W_1 + b_1) * W_2 + b_2$; dims: $d \rightarrow 4d \rightarrow d$	Stores world knowledge; applied per position
BatchNorm	$\hat{x} = (x - \mu) / \sqrt{\sigma^2 + \epsilon}$; $y = \gamma * \hat{x} + \beta$	At inference: use running EMA, not batch stats
Bradley-Terry	$P(y_w > y_l) = \sigma(r_w - r_l)$	Maximize $\log \sigma(r_w - r_l)$ during RM training
RLHF Objective	$E[r(x, y)] - \beta * KL(\pi_{\theta} \pi_{\text{ref}})$	KL is vs π_{ref} (SFT model), NOT the reward model
PPO-CLIP	$E[\min(r_t * A_t, \text{clip}(r_t, 1 - \epsilon, 1 + \epsilon) * A_t)]$	No gradient outside $[1 - \epsilon, 1 + \epsilon]$ trust region
LoRA	$W' = W + B * A$; $r \ll \min(d, k)$	~0.1% params trained; merge at inference for no latency

Most Common Exam Traps

Trap	Wrong Assumption	Correct Answer
GloVe vs word2vec	GloVe uses negative sampling	GloVe uses NONE; minimises weighted LS on log co-occurrence
KL in RLHF	KL is vs the reward model	KL is vs π_{ref} (frozen SFT model)
RLHF KL penalty	KL is maximised $\rightarrow$ be different from reference	KL is penalised $\rightarrow$ stay CLOSE to π_{ref}
Bahdanau state	Uses current decoder state s_t	Uses PREVIOUS state s_{t-1} (context feeds into s_t)
Luong state	Uses previous state s_{t-1}	Uses CURRENT state s_t (computed first, then attends)
BERT for generation	BERT can generate text like GPT	BERT is bidirectional; MLM not designed for generation
Beam search	Larger beam $\rightarrow$ lower probability	Larger beam $\rightarrow$ at least as high probability (monotone)
Dropout in CNNs	Standard dropout is fine for conv layers	Weak due to spatial correlation; use SpatialDropout/DropBlock
RoIPool vs RoIAlign	Both give same spatial precision	RoIPool quantizes $\rightarrow$ misalignment $\rightarrow$ bad for masks
T5 vs BERT	T5 masks individual tokens like BERT	T5 masks SPANS with sentinel tokens
Self-Refine vs Reflexion	Both improve across multiple trials	Self-Refine: single trial only; Reflexion: cross-trial memory

Trap	Wrong Assumption	Correct Answer
RAG-Token for legal QA	RAG-Token is safer (more information)	RAG-Token mixes docs per token -> contradictions
PPO-CLIP gradient	Gradient always proportional to $rt \cdot \hat{A}_t$	When rt outside trust region, gradient = 0 from rt

Architecture Quick Reference

Model	Architecture	Pre-training	Task
BERT	Encoder-only	MLM + NSP	Classification, NER, extractive QA
GPT	Decoder-only	Causal LM	Generation, instruction following
T5	Encoder-Decoder	Span corruption	Any seq2seq (translation, summarisation)
BART	Encoder-Decoder	Multi-noise denoising	Summarisation, generation, translation
LLaMA	Decoder-only	Causal LM (RoPE+RMSNorm)	Open-source generation/instruction following

Key Numerical Benchmarks

- BERT-base: $d=768$, $h=12$ heads, $L=12$ layers, $\sim 110M$ parameters
- BERT-large: $d=1024$, $h=16$ heads, $L=24$ layers, $\sim 340M$ parameters
- Transformer per-layer params: $\sim 12d^2$ ($4d^2$ MHA + $8d^2$ FFN)
- BERT-base: $12 \times 12 \times 768^2 \sim 85M$ params + $23.8M$ embeddings $\sim 110M$ total
- Bradley-Terry: $\sigma(1.5) \sim 0.818$; $\sigma(2.0) \sim 0.880$; $\sigma(3.0) \sim 0.953$
- LoRA typical rank: $r=4, 8, \text{ or } 16 \rightarrow 0.01\text{-}0.1\%$ of parameters trained
- FiD vs concat cost: $O(K \cdot L^2)$ vs $O((K \cdot L)^2)$ $\rightarrow K$ times faster for K docs

Answer Templates for Exam Questions

True/False Template

Structure: '[True/False]. [State the correct fact]. [Why the opposite is wrong, if relevant].'
 Example: 'False. GloVe does not use any negative sampling. It minimises weighted least squares on log co-occurrence counts from the global co-occurrence matrix — the key difference from word2vec SGNS.'

MCQ Template

Structure: '[Letter]. [Why this option is correct in 1-2 sentences]. [Optionally: briefly note why the most tempting wrong answer is incorrect].'

Derivation/Explanation Template

Structure: (1) State what it is/does in one sentence. (2) Write the mathematical form. (3) Explain each symbol. (4) Give the intuition or a concrete example. (5) State a key property or limitation. Always define every symbol — examiners give marks for correct notation.

Design Question Template

Structure: Stage 1 (data/base model) $\rightarrow$ Stage 2 (SFT) $\rightarrow$ Stage 3 (reward model) $\rightarrow$ Stage 4 (RLHF/alignment). Then deployment: retrieval $\rightarrow$ safety check $\rightarrow$ generation $\rightarrow$ confidence $\rightarrow$ fallback. Always name specific algorithms (LoRA, PPO, Bradley-Terry, DPR, etc.).

Deep Learning & GenAI — IIT Bombay ePGD AI & DS | May 2026 End Semester

Source files: 01_Detailed_Concepts_and_Math.pdf | 03_Mock_Paper_with_Solutions.pdf | DL_FormulaGuide | DL_MasterCheatSheet |
DL_SampleExams | ePGD_End_term_Model_Paper